30.09.2026

BSI IT-Grundschutz

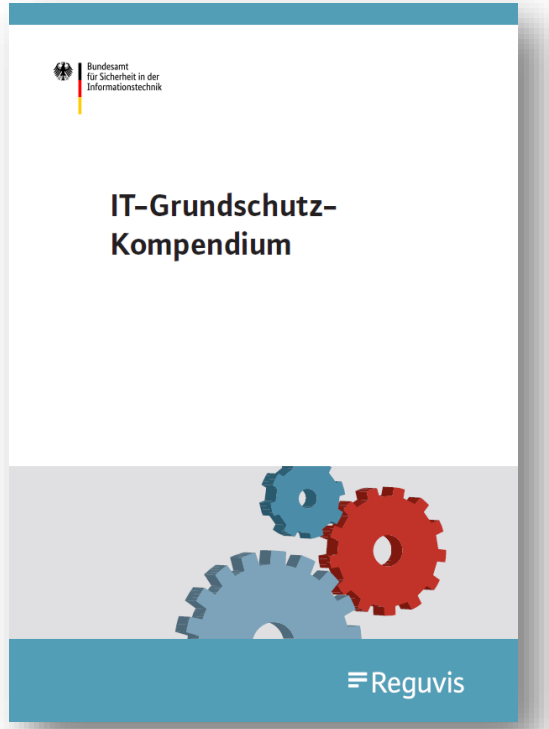
Was ist der BSI IT-Grundschutz?

Der IT-Grundschutz ist ein methodisches Rahmenwerk des Bundesamts für Sicherheit in der Informationstechnik (BSI).

Es unterstützt Organisationen dabei, ein angemessenes Sicherheitsniveau für ihre IT systematisch zu erreichen.

Was ist der BSI IT-Grundschutz?

Statt für jedes Unternehmen individuell von Grund auf zu überlegen, welche Sicherheitsmaßnahmen nötig sind, bietet der IT-Grundschutz einen strukturierten Katalog von Bausteinen mit konkreten und bewährten Maßnahmen für typische IT-Komponenten (von Servern über Netzwerke bis hin zu organisatorischen Prozessen).



Aufgabe

- Sie laden sich das Dokument beim BSI herunter.
- Sie schmökern ein bisschen darin.

Sie müssen es nicht komplett lesen 😇



IT-Grundschutz-Kompendium

Das IT-Grundschutz-Kompendium ist die zentrale Sammlung von Empfehlungen des Bundesamts für Sicherheit in der Informationstechnik zur Absicherung von IT-Systemen.

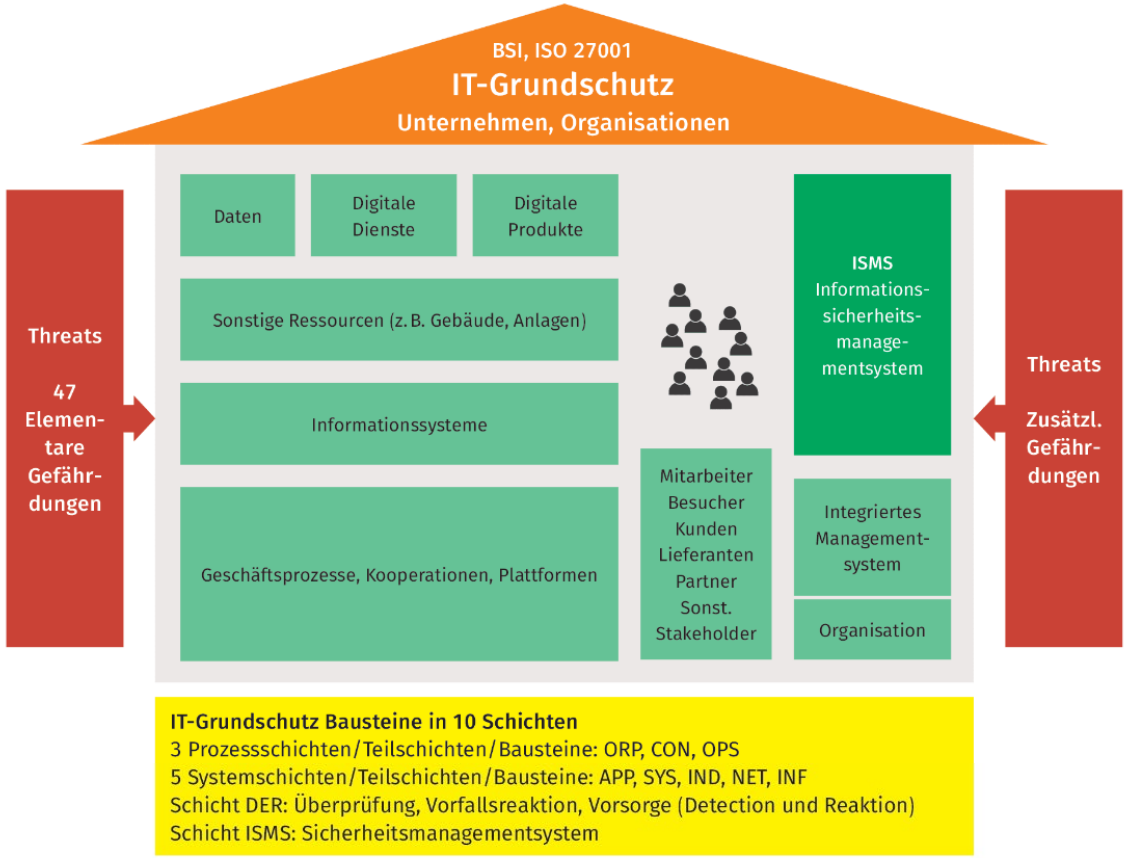


Inhalt:

- Bausteine für typische IT-Objekte (z. B. Server, Netze, Anwendungen, Organisation).
- Zu jedem Baustein: Bedrohungen & konkrete Sicherheitsmaßnahmen.
- Grundlage für den Aufbau eines (ISMS) nach IT-Grundschutz.

IT-Grundschutz-Kompendium





Was ist der BSI IT-Grundschutz?

Der IT-Grundschutz ist insbesondere für Behörden und öffentliche Einrichtungen in Deutschland relevant, wird aber auch in der Privatwirtschaft als anerkannter Standard für Informationssicherheit genutzt.

Die Schutzbedarfsanalyse: Der erste Schritt

- Bevor Sicherheitsmaßnahmen festgelegt werden können, muss geklärt werden, wie schützenswert die jeweiligen Daten und Systeme überhaupt sind, denn nicht jede Information benötigt das gleiche Schutzniveau.
- Die Schutzbedarfsanalyse ist der methodische Schritt, bei dem für jeden relevanten Bereich (etwa eine Anwendung, ein System oder einen Datenbestand) bewertet wird, welche Konsequenzen ein Sicherheitsvorfall hätte.

Die Schutzbedarfsanalyse: Der erste Schritt

Dabei werden klassischerweise drei Schutzziele betrachtet:

- **Vertraulichkeit**
  Unbefugte dürfen die Daten nicht einsehen können.
- **Integrität**
  Daten dürfen nicht unbemerkt verändert werden.
- **Verfügbarkeit**
  Daten und Systeme müssen zugänglich sein, wenn sie benötigt werden.



Dies wird oft als CIA-Triade bezeichnet (Confidentiality, Integrity, Availability).

Die drei Schutzbedarfskategorien

Für jedes der drei Schutzziele wird der Schutzbedarf in eine von drei Kategorien eingeordnet:

- „Normal“ bedeutet, dass die Auswirkungen eines Sicherheitsvorfalls begrenzt und überschaubar wären (etwa wenn ein interner Prozess kurzzeitig verzögert wird).
- „Hoch“ bedeutet, dass die Auswirkungen erheblich wären (beträchtliche finanzielle Verluste, ein deutlicher Reputationsschaden, oder Verstöße gegen gesetzliche Vorgaben).
- „Sehr hoch“ bedeutet, dass die Auswirkungen existenzbedrohend für die Organisation wären oder sogar Gefahr für Leib und Leben bestehen könnte (etwa bei Systemen, die kritische Infrastrukturen wie Krankenhäuser oder Energieversorgung betreffen).

Die drei Schutzbedarfskategorien

Wichtig:

Die Einstufung erfolgt getrennt für Vertraulichkeit, Integrität und Verfügbarkeit.

So kann ein System beispielsweise einen hohen Schutzbedarf bei der Verfügbarkeit haben, aber nur normalen Schutzbedarf bei der Vertraulichkeit, wenn die verarbeiteten Daten nicht besonders sensibel, ihre ständige Erreichbarkeit aber geschäftskritisch ist.

Grundschutz-Bausteine: Das Baukastenprinzip

- Nachdem der Schutzbedarf ermittelt wurde, kommen die Grundschutz-Bausteine ins Spiel.
- Ein Baustein ist eine in sich geschlossene Sammlung von Anforderungen und Maßnahmen für einen bestimmten Bereich der IT (etwa für Server allgemein, für Windows-Systeme spezifisch, für Netzwerkkomponenten, für Rechenzentren) oder für organisatorische Themen wie Personalmanagement und Notfallvorsorge.

Prozess-Bausteine

ISMS – Sicherheitsmanagement
ORP – Organisation und Personal
CON – Konzepte und Vorgehensweisen
OPS – Betrieb
DER – Detektion und Reaktion

System-Bausteine

APP – Anwendungen
SYS – IT-Systeme
IND – Industrielle IT
NET – Netze und Kommunikation
INF – Infrastruktur

Grundschutz-Bausteine: Das Baukastenprinzip

- Jeder Baustein beschreibt, welche Gefährdungen in diesem Bereich typischerweise bestehen und welche Anforderungen erfüllt werden sollten, um diesen Gefährdungen zu begegnen.
- Das Baukastenprinzip ermöglicht es, ein Unternehmen modular zu betrachten: Für jede vorhandene IT-Komponente wird der passende Baustein herangezogen, und aus der Summe aller relevanten Bausteine ergibt sich ein vollständiges Sicherheitskonzept für die gesamte Organisation.

Vom Schutzbedarf zur Maßnahme

Schutzbedarfsanalyse und Bausteine greifen zusammen:

- Die Schutzbedarfsanalyse bestimmt, wie kritisch ein bestimmter Bereich ist.
- Die Bausteine liefern die passenden Maßnahmen für diesen Bereich.

Bei normalem Schutzbedarf reichen häufig die grundlegenden, im jeweiligen Baustein beschriebenen Standardmaßnahmen aus. Bei hohem oder sehr hohem Schutzbedarf müssen zusätzlich zu den Basismaßnahmen weitergehende, individuell zugeschnittene Maßnahmen ergriffen werden. Der IT-Grundschutz spricht hier von einer ergänzenden Risikoanalyse, die über die Standardmaßnahmen hinausgeht.

Vom Schutzbedarf zur Maßnahme

Das Ergebnis dieses Prozesses ist ein dokumentiertes Sicherheitskonzept, das für jeden Bereich der IT begründet, welche Maßnahmen warum umgesetzt werden.

Angepasst an die tatsächliche Kritikalität, nicht pauschal für alle Systeme gleich.

Beispiel zur Veranschaulichung

Ein Unternehmen betreibt zwei Server: einen internen Fileserver für allgemeine Bürodokumente und einen Server, der Patientendaten einer angeschlossenen Arztpraxis verarbeitet.

- Für den Fileserver ergibt die Schutzbedarfsanalyse bei allen drei Schutzzielen die Kategorie „normal“.
- Die im Server-Baustein beschriebenen Standardmaßnahmen, etwa regelmäßige Backups und Zugriffsbeschränkungen, sind ausreichend.

Beispiel zur Veranschaulichung

- Für den Server mit Patientendaten ergibt die Analyse bei Vertraulichkeit und Verfügbarkeit die Kategorie „hoch“ oder möglicherweise „sehr hoch“.
- Ein unbefugter Zugriff auf Gesundheitsdaten hätte erhebliche rechtliche und persönliche Konsequenzen, ein Ausfall könnte die medizinische Versorgung gefährden.
- Für diesen Server reichen die Standardmaßnahmen des Bausteins nicht aus: zusätzliche Maßnahmen wie Verschlüsselung, redundante Auslegung und strengere Zugriffskontrollen werden erforderlich.

Prüfungsrelevante Einordnung



In der AP2 wird der IT-Grundschutz meist konzeptionell geprüft. Es wird nicht erwartet, konkrete Bausteine oder deren Nummerierung zu kennen. Die grundlegende Methodik sollte aber verstanden sein und angewendet werden können.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen möchte seine IT-Systeme nach dem BSI IT-Grundschutz absichern. Erläutern Sie das Konzept der Schutzbedarfs-analyse und ordnen Sie ein, welchen Schutzbedarf ein Server mit Kundendaten hinsichtlich Vertraulichkeit haben könnte.“

Prüfungsrelevante Einordnung



Die erwartete Antwort erklärt, dass die Schutzbedarfsanalyse die Auswirkungen eines Sicherheitsvorfalls hinsichtlich Vertraulichkeit, Integrität und Verfügbarkeit bewertet.

Sie ordnet diese in die drei Kategorien „normal“ / „hoch“ / „sehr hoch“ ein, und begründet für das Beispiel (Kundendaten unterliegen der DSGVO, ein unbefugter Zugriff hätte rechtliche Konsequenzen, daher mindestens „hoch“).

Wer zusätzlich erklärt, dass aus dem ermittelten Schutzbedarf die passenden Maßnahmen über die Grundschutz-Bausteine abgeleitet werden, zeigt das vollständige Konzept.

Themenblock 8
Lizenzmanagement und Compliance

Überblick

- Lizenzmodelle: Named User, Concurrent User, Core-basiert, Subscription vs. Perpetual
- Open Source Lizenztypen (GPL, MIT, Apache)
- Software Asset Management (SAM): Pflicht und Risiken bei Nichteinhaltung
- EULA-Grundverständnis

Lizenzmodelle

Warum Lizenzmodelle prüfungsrelevant sind

Software wird selten einfach „gekauft“.

- Stattdessen wird Software lizenziert.
- Die Art der Lizenzierung bestimmt erheblich, wie viel ein Unternehmen dafür bezahlen muss und wie flexibel es die Software nutzen kann.

Warum Lizenzmodelle prüfungsrelevant sind

Für IT-Verantwortliche ist das Verständnis von Lizenzmodellen aus zwei Gründen wichtig:

- Um bei der Beschaffung das wirtschaftlich passende Modell zu wählen.
- Um Compliance-Risiken zu vermeiden (die Nutzung von Software außerhalb der lizenzierten Bedingungen ist ein Lizenzverstoß, unabhängig davon, ob er absichtlich oder versehentlich erfolgt).

„Named User“ Lizenzierung

Bei der Named-User-Lizenzierung wird die Software für bestimmte, namentlich benannte Personen lizenziert.

- Jede Lizenz ist einem konkreten Benutzer zugeordnet, und zwar unabhängig davon, wie oft oder wie intensiv diese Person die Software tatsächlich nutzt.
- Ein Unternehmen mit 50 Mitarbeitern, die alle Zugriff auf eine bestimmte Software benötigen, müsste demnach 50 Named-User-Lizenzen erwerben, auch wenn nicht alle 50 die Software gleichzeitig oder täglich nutzen.

„Named User“ Lizenzierung

Vorteil: Einfachheit und Planbarkeit

- Jeder Benutzer hat garantiert jederzeit Zugriff, ohne Wartezeit oder Konflikte mit anderen Nutzern.

Nachteil: mangelnde Flexibilität bei schwankender Nutzung

- Wird eine Lizenz einem Mitarbeiter zugewiesen, der die Software nur selten braucht, ist diese Lizenz für andere blockiert, selbst wenn sie ungenutzt bleibt.

„Concurrent User“ Lizenzierung

Bei der Concurrent-User-Lizenzierung („Floating License“) wird nicht festgelegt, wer die Software nutzen darf, sondern wie viele Personen sie gleichzeitig nutzen dürfen.

- Ein Unternehmen mit 50 Mitarbeitern, von denen aber im Schnitt nie mehr als 15 gleichzeitig auf die Software zugreifen, könnte mit 15 Concurrent-User-Lizenzen auskommen.
- Greift ein 16. Mitarbeiter zu, während alle 15 Lizenzen belegt sind, erhält er entweder eine Fehlermeldung oder muss warten, bis eine Lizenz frei wird.

„Concurrent User“ Lizenzierung

- Dieses Modell eignet sich besonders für Software, die nicht von allen Mitarbeitern dauerhaft, sondern nur gelegentlich oder zeitversetzt genutzt wird, etwa spezialisierte Fachanwendungen, die nur bei Bedarf geöffnet werden.

„Core-basierte“ Lizenzierung

Die core-basierten Lizenzierung wird vor allem bei Server-Software wie Datenbanksystemen genutzt.

- Der Lizenzpreis richtet sich nach der Anzahl der CPU-Kerne (Cores) des Servers, auf dem die Software läuft und nicht nach der Anzahl der Benutzer.
- Die Begründung dahinter: Die Leistungsfähigkeit eines Servers (und damit potenziell auch der wirtschaftliche Nutzen der darauf laufenden Software) korreliert mit der verfügbaren Rechen-leistung.

„Core-basierte“ Lizenzierung

Für Unternehmen hat das eine wichtige praktische Konsequenz im Kontext von Virtualisierung:

- Wird eine core-lizenzierte Software auf einer VM betrieben, muss je nach Lizenzbedingungen unter Umständen für alle physischen Kerne des Hosts lizenziert werden, auf dem die VM theoretisch laufen könnte und nicht nur für die der VM zugewiesenen virtuellen Kerne.
- Das kann bei Virtualisierungsentscheidungen erhebliche, leicht übersehene Kostenfolgen haben.

„Subscription“ vs. „Perpetual License“

Eine Perpetual License (also eine "klassische" oder dauerhafte Lizenz) wird einmalig erworben und berechtigt zur unbefristeten Nutzung der erworbenen Softwareversion.

- Updates und neue Versionen sind oft nicht automatisch enthalten, sondern müssen gegebenenfalls separat erworben werden.

„Subscription“ vs. „Perpetual License“

Eine Subscription (ein Abonnement) berechtigt zur Nutzung der Software nur für einen begrenzten Zeitraum, typischerweise monatlich oder jährlich abgerechnet.

- Subscriptions enthalten in der Regel automatisch alle Updates und neuen Versionen während der Laufzeit.
- Mit dem Ende des Abonnements endet auch das Nutzungsrecht.

„Subscription“ vs. „Perpetual License“

Dieser Unterschied verbindet sich direkt mit dem bereits behandelten Capex-vs-Opex-Konzept:

- Perpetual Licenses sind tendenziell Capex-ähnlich (einmalige hohe Investition).
- Subscriptions sind Opex-ähnlich (laufende, planbare Kosten).

Vergleich und Entscheidungslogik

Für die Wahl des passenden Lizenz-modells lassen sich folgende Leitfragen formulieren:

- Named User vs. Concurrent User
- Core-basierte Lizenzierung
- Subscription vs. Perpetual

Wie viele Nutzer benötigen potenziell Zugriff, und wie viele nutzen die Software tatsächlich gleichzeitig?

Ist die Differenz groß, ist Concurrent User wirtschaftlicher.

Vergleich und Entscheidungslogik

Für die Wahl des passenden Lizenzmodells lassen sich folgende Leitfragen formulieren:

- Named User vs. Concurrent User
- Core-basierte Lizenzierung
- Subscription vs. Perpetual

Welche Hardware-Konfiguration liegt zugrunde, und welche Konsequenzen hat eine Virtualisierung für die Lizenzkosten?

Vergleich und Entscheidungslogik

Für die Wahl des passenden Lizenz-modells lassen sich folgende Leitfragen formulieren:

- Named User vs. Concurrent User
- Core-basierte Lizenzierung
- Subscription vs. Perpetual

Sofern die Software langfristig und in unveränderter Form genutzt wird ist eher Perpetual sinnvoll.

Ist eine kontinuierliche Aktualisierung wichtig und soll die Kostenstruktur planbar und ohne große Einmalinvestition bleiben, führt das eher zu Subscription.

Prüfungsrelevante Einordnung



In der AP2 wird meist ein Unternehmensszenario mit konkreten Nutzungszahlen beschrieben, und der Prüfling soll ein geeignetes Lizenzmodell empfehlen und begründen.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen mit 80 Mitarbeitern setzt eine Spezialsoftware ein, die laut Auswertung von maximal 20 Personen gleichzeitig genutzt wird. Welches Lizenzmodell empfehlen Sie, und warum?“

Prüfungsrelevante Einordnung



Die erwartete Antwort empfiehlt Concurrent-User-Lizenzierung mit etwa 20 Lizenzen, begründet dies mit der deutlichen Kostenersparnis gegenüber 80 Named-User-Lizenzen, und weist auf den möglichen Nachteil hin. Bei gleichzeitigem Bedarf von mehr als 20 Nutzern könnten Wartezeiten entstehen.

Wer zusätzlich erkennt, dass bei stark schwankendem Bedarf auch die Größe des Lizenzpools regelmäßig überprüft werden sollte, zeigt vertieftes Verständnis.

Open Source Lizenztypen

Einleitung

- Open-Source-Software ist heute in praktisch jeder Unternehmensumgebung im Einsatz, oft ohne dass es bewusst wahrgenommen wird, da viele kommerzielle Produkte Open-Source-Komponenten enthalten.
- „Open Source“ bedeutet jedoch nicht „ohne Bedingungen nutzbar“.
- Jede Open-Source-Software unterliegt einer Lizenz, die festlegt, was mit dem Code erlaubt ist und was nicht.

Einleitung

Für Unternehmen ist das vor allem relevant, wenn sie Open-Source-Software nicht nur intern nutzen, sondern in eigene Produkte einbauen, die sie dann weitergeben.

„Copyleft“ vs. „Permissiv“

Open-Source-Lizenzen lassen sich grob in zwei Kategorien einteilen.

Diese unterscheiden sich vor allem darin, was passiert, wenn man den Code verändert oder in eigene Software einbaut.

- Copyleft-Lizenzen
- Permissive Lizenzen

„Copyleft“ vs. „Permissiv“

Copyleft-Lizenzen verlangen, dass abgeleitete Werke (also Software, die den Open-Source-Code enthält oder darauf basiert) unter denselben Lizenzbedingungen weitergegeben werden müssen.

Wer also Code unter einer Copyleft-Lizenz in eigene Software einbaut und diese weitergibt, muss auch den eigenen Code unter derselben Lizenz offenlegen.

„Copyleft“ vs. „Permissiv“

Permissive Lizenzen sind deutlich weniger restriktiv.

Sie erlauben die Nutzung, Veränderung und Weitergabe des Codes auch in proprietärer, geschlossener Software ohne die Verpflichtung, eigenen Code offenzulegen.

„GPL“ (=Copyleft)

Die GPL (GNU General Public License) ist die bekannteste und am weitesten verbreitete Copyleft-Lizenz.

- Software unter GPL darf frei genutzt, verändert und weitergegeben werden.
- Zentrale Auflage: jede Weitergabe, auch in veränderter oder erweiterter Form, muss ebenfalls unter der GPL erfolgen und der Quellcode muss mitgeliefert werden.

„GPL“ (= Copyleft)

Für Unternehmen ist die GPL besonders relevant, wenn sie GPL-lizenzierten Code in ein eigenes Produkt integrieren wollen, das sie kommerziell vertreiben, denn dann müssten sie unter Umständen den Quellcode ihres gesamten Produkts offenlegen.

Das macht GPL-Code für viele Unternehmen im Produktkontext zu einem bewusst zu prüfenden Risiko, während die reine interne Nutzung von GPL-Software meist unproblematisch ist.

„MIT“ (= Permissiv)

Die MIT-Lizenz ist eine der einfachsten und freizügigsten Open-Source-Lizenzen.

- Sie erlaubt nahezu uneingeschränkte Nutzung, Veränderung und Weitergabe (auch in proprietärer Software).
- Im Wesentlichen verlangt sie nur, dass der ursprüngliche Lizenz- und Copyright-Hinweis erhalten bleibt.

„Apache“ (= Permissiv)

Die Apache-Lizenz (Version 2.0) ist ähnlich permissiv wie MIT, enthält aber zusätzlich Regelungen zu Patenten.

- Sie gewährt Nutzern explizit eine Patentlizenz für die im Code enthaltenen Technologien.
- Damit schützt sie vor späteren Patentstreitigkeiten durch die ursprünglichen Entwickler.

„MIT“ und „Apache“ (= Permissiv)

Beide Lizenzen sind aus Unternehmenssicht unkompliziert: Code unter MIT oder Apache kann in eigene, auch proprietäre Produkte eingebaut werden, ohne dass der eigene Code offengelegt werden muss.

Praktische Konsequenzen für Unternehmen

Für ein Unternehmen, das Software entwickelt oder Open-Source-Komponenten in eigene Produkte integriert, ergibt sich aus diesen Lizenztypen eine konkrete Handlungsanforderung:

- Vor der Integration einer Open-Source-Komponente muss geprüft werden, unter welcher Lizenz sie steht und ob diese Lizenz mit dem geplanten Einsatzzweck vereinbar ist.

- Permissive Lizenzen wie MIT oder Apache stellen in der Regel kein Problem dar.
- Copyleft-Lizenzen wie die GPL erfordern eine genauere Prüfung, insbesondere, wenn die eigene Software nicht selbst als Open Source weitergegeben werden soll.

Praktische Konsequenzen für Unternehmen

Größere Unternehmen führen häufig automatisierte Scans ihres Codes durch, um unbeabsichtigt eingebundene Lizenzen mit Copyleft-Verpflichtungen frühzeitig zu erkennen und spätere Streitigkeiten zu vermeiden.

Prüfungsrelevante Einordnung



In der AP2 wird das Thema meist auf der Ebene der grundlegenden Unterscheidung geprüft (copyleft versus permissiv) und seltener mit Detailwissen zu einzelnen Lizenzen.

Eine typische Aufgabe könnte lauten:

„Ein Unternehmen möchte eine Open-Source-Bibliothek in sein kommerzielles, nicht offen verfügbares Softwareprodukt integrieren. Erklären Sie, worauf bei der Lizenzwahl der Bibliothek geachtet werden muss.“

Prüfungsrelevante Einordnung



Die erwartete Antwort erklärt den Unterschied zwischen Copyleft- und permissiven Lizenzen. Sie nennt GPL als Beispiel für eine Lizenz, die bei Weitergabe die Offenlegung des eigenen Codes verlangen könnte, und MIT oder Apache als Beispiele für unproblematische, permissive Lizenzen für diesen Einsatzzweck.

Wer erklärt, warum die Wahl der Lizenz hier eine wirtschaftliche und rechtliche Konsequenz hat (nämlich die mögliche Offenlegungspflicht des eigenen Quellcodes) zeigt das relevante Verständnis, ohne dass Detailkenntnisse einzelner Lizenztexte nötig wären.

Software Asset Management

Was ist Software Asset Management?

Als Software Asset Management (SAM) bezeichnet man die systematische Verwaltung und Überwachung aller Softwarelizenzen, die ein Unternehmen besitzt und einsetzt.

Was ist Software Asset Management?

Im Kern geht es um eine einfache, aber in der Praxis oft vernachlässigte Frage:

- Stimmt die Anzahl der tatsächlich genutzten Softwareinstallationen mit der Anzahl der erworbenen Lizenzen überein?

SAM ist damit die organisatorische Antwort auf die Vielzahl unterschiedlicher Lizenzmodelle, die wir zuvor behandelt haben.

Denn je komplexer die Lizenzmodelle (Named User, Concurrent User, Core-basiert), desto schwieriger wird es, ohne systematische Verwaltung den Überblick zu behalten.

Was SAM konkret umfasst

SAM beginnt mit einer vollständigen Bestandsaufnahme:

- Welche Software ist auf welchen Systemen installiert?
- Welche Lizenzen wurden dafür erworben?

Diese Inventarisierung erfolgt in größeren Umgebungen meist automatisiert durch spezialisierte Software, die alle installierten Programme auf allen Endgeräten und Servern erfasst.

Was SAM konkret umfasst

Auf Basis dieser Inventarisierung wird die Ist-Situation (tatsächliche Nutzung) mit der Soll-Situation (erworbene Lizenzen) verglichen.

SAM umfasst dabei nicht nur die reine Zählung, sondern auch die Verwaltung der Lizenzbedingungen selbst (also welche Nutzungsrechte mit welcher Lizenz verbunden sind, wann Lizenzen ablaufen oder erneuert werden müssen, und welche Lizenzmodelle für welche Software gelten).

Unter- und Überlizenzierung

Aus dem Vergleich von Ist- und Soll-Zustand ergeben sich zwei mögliche Abweichungen:

- Unterlizenzierung
- Überlizenzierung

Beide sind problematisch, aber auf unterschiedliche Weise.

Unter- und Überlizenzierung

Unterlizenzierung

Es gibt mehr Installationen oder Nutzer vorhanden sind, als Lizenzen erworben wurden.

Das Unternehmen nutzt also Software, für die es nicht ausreichend bezahlt hat, und verstößt damit gegen die Lizenzbedingungen.

Unter- und Überlizenzierung

Überlizenzierung

Das Unternehmen hat mehr Lizenzen erworben, als tatsächlich genutzt werden.

Es zahlt also für etwas, das es nicht braucht.

Unter- und Überlizenzierung

Überlizenzierung ist „nur“ ein wirtschaftliches Problem, denn es entstehen unnötige Kosten.

Unterlizenzierung ist ein rechtliches Problem: ein Lizenzverstoß, unabhängig davon, ob er absichtlich oder durch mangelnde Übersicht entstanden ist.

Risiken bei Nichteinhaltung

Software-Audit

Softwarehersteller behalten sich vertraglich das Recht vor, die Einhaltung der Lizenzbedingungen zu überprüfen.

Wird im Rahmen eines solchen Audits eine Unterlizenzierung festgestellt, drohen dem Unternehmen erhebliche Konsequenzen:

- Nachzahlungen für die fehlenden Lizenzen, häufig zu regulären (nicht rabattierten) Listenpreisen.
- Vertragsstrafen, die deutlich über den ursprünglichen Lizenzkosten liegen können.

Risiken bei Nichteinhaltung

- Neben den finanziellen Risiken entsteht auch ein Reputationsrisiko, wenn ein Unternehmen öffentlich als Lizenzverstoßer bekannt wird, etwa im Rahmen einer rechtlichen Auseinandersetzung.
- Aus rechtlicher Sicht stellt die Nutzung von Software ohne ausreichende Lizenzierung außerdem eine Urheberrechtsverletzung dar.

SAM als kontinuierlicher Prozess

SAM ist keine einmalige Aufgabe (nur eine Bestandsaufnahme bei der Einführung neuer Software reicht auf Dauer nicht aus).

SAM ist ein kontinuierlicher Prozess, weil sich die Ist-Situation ständig ändert:

- Neue Mitarbeiter erhalten neue Software
- Mitarbeiter verlassen das Unternehmen, ohne dass ihre Lizenzen freigegeben werden
- Neue Server werden aufgesetzt
- Cloud-Ressourcen werden flexibel skaliert

SAM als kontinuierlicher Prozess

Ohne kontinuierliches SAM driften Ist- und Soll-Zustand mit der Zeit zwangsläufig auseinander. Das passiert oft unbemerkt, bis ein Audit die Differenz aufdeckt.

Effektives SAM bedeutet daher:

- Regelmäßige automatisierte Inventarisierung
- Regelmäßiger Abgleich mit den Lizenzverträgen
- Definierte Prozesse für die Lizenzvergabe
- Lizenzrückgabe bei Personalveränderungen

Prüfungsrelevante Einordnung



In der AP2 wird SAM meist im Kontext eines Szenarios geprüft, in dem ein Unternehmen seinen Lizenzbestand nicht systematisch verwaltet hat.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen hat in den letzten Jahren mehrfach Mitarbeiter eingestellt und Software lizenziert, ohne den Überblick über den aktuellen Lizenzbestand zu behalten. Erläutern Sie, welches Risiko dadurch entsteht und wie diesem Risiko durch ein geeignetes Konzept begegnet werden kann.“

Prüfungsrelevante Einordnung



Die erwartete Antwort benennt das Risiko der Unterlizenzierung und die Konsequenzen eines Software-Audits (Nachzahlungen und Vertragsstrafen).

Sie schlägt die Einführung von Software Asset Management vor und zeigt die Vorteil: automatisierte Inventarisierung, regelmäßiger Abgleich mit den Lizenzverträgen, und definierte Prozesse für Lizenzvergabe und -entzug.

Wer den Unterschied zwischen Unter- und Überlizenzierung klar benennt, zeigt, dass SAM nicht nur ein Compliance-Thema, sondern auch ein wirtschaftliches Optimierungsthema ist.

EULA-Grundverständnis

Was ist eine EULA?

EULA steht für End User License Agreement (Endbenutzer-Lizenzvertrag). Es handelt sich um die vertragliche Vereinbarung zwischen einem Softwarehersteller und der Person oder Organisation, die die Software nutzt.

Was ist eine EULA?

Die EULA regelt, was mit der Software erlaubt ist und was nicht, sie ist quasi die "Gebrauchsanweisung" in rechtlicher Form.

- Lizenzmodelle wie Named User oder Concurrent User beschreiben, wie viele Personen die Software nutzen dürfen.
- Open-Source-Lizenzen wie GPL oder MIT regeln, was mit dem Quellcode gemacht werden darf.
- EULA ist der übergeordnete rechtliche Rahmen, der bei kommerzieller Software die Nutzungsbedingungen insgesamt festlegt (typischerweise inklusive der Lizenzmodelle und etwaiger Einschränkungen).

Typische Inhalte einer EULA

EULAs enthalten in der Regel mehrere wiederkehrende Bestandteile:

Erlaubte Nutzungsformen:

- Wie viele Geräte, Nutzer oder Installationen sind abgedeckt?
- Darf die Software privat oder kommerziell genutzt werden?

Typische Inhalte einer EULA

Einschränkungen:

- Verbote des Reverse Engineering (also der Versuch, aus dem fertigen Programm den Quellcode zu rekonstruieren).
- Verbote der Weitergabe oder Unterlizenzierung an Dritte.
- Haftungsausschlüsse: Hersteller schließen typischerweise jede Haftung für Schäden aus, die durch die Nutzung der Software entstehen.
- Regelungen zur Vertragslaufzeit, zu Updates und zur Beendigung der Lizenz bei Vertragsverstoß.

Wie EULAs wirksam werden

- EULAs werden typischerweise durch eine aktive Zustimmung des Nutzers wirksam.
- Häufig geschieht durch das Anklicken einer "Ich akzeptiere"-Schaltfläche bei der Installation oder dem ersten Start der Software.
- Diese Zustimmung gilt rechtlich als Vertragsabschluss, auch wenn die meisten Nutzer die oft sehr langen Texte nicht vollständig lesen.

Wie EULAs wirksam werden

Für Unternehmen ist relevant:

- Diese Zustimmung ist in der Regel für jede Installation einzeln erforderlich.
- Die Bedingungen der EULA sind für ein Unternehmen genauso bindend sind wie für Privatpersonen.

Wird Software in einem Unternehmen zentral ausgerollt, sollte daher vorab geprüft werden, ob die EULA-Bedingungen mit dem geplanten Einsatzszenario vereinbar sind (etwa hinsichtlich der Anzahl der Installationen oder der Frage, ob eine kommerzielle Nutzung überhaupt erlaubt ist, wenn eine Software ursprünglich für den Privatgebrauch lizenziert wurde).

Prüfungsrelevante Einordnung



In der AP2 wird die EULA meist als Begriff im Zusammenhang mit Lizenzmanagement und Compliance abgefragt.

Es geht weniger um juristische Details als um das grundlegende Verständnis, dass jede Software einem rechtlich bindenden Nutzungsvertrag unterliegt.

Eine typische Frage könnte lauten:
„Was versteht man unter einer EULA, und welche Bedeutung hat sie für ein Unternehmen bei der Beschaffung von Software?“

Prüfungsrelevante Einordnung



Die erwartete Antwort erklärt, dass die EULA der Endbenutzer-Lizenzvertrag ist, der die Nutzungsbedingungen einer Software festlegt (erlaubte Nutzungsformen, Einschränkungen und Haftungsausschlüsse) und dass diese Bedingungen für das Unternehmen rechtlich bindend sind, sobald sie akzeptiert werden.

Im Kontext von Compliance und SAM ist wichtig zu erkennen, dass die EULA die Grundlage dafür bildet, was als „ordnungsgemäße Lizenzierung“ überhaupt gilt. Sie ist also die Referenz, an der bei einem Software-Audit gemessen wird.

30.09.2026

Ende